

Session 20 (AIML) - Assignment Question

- **Name of Student** : Shruti Dnyaneshwar Darwatkar
- **Domain Name**: AIML Development
- **College Name**: Zeal Polytechnic , Narhe
- **Branch Name**: AIML
- **Github url**: https://github.com/shrutidarwatkar-sketch/Task_ITR.git

Q1. Preparing Features)

Load your cleaned Ford Car Dataset.

- Separate the dataset into Independent Features X and Dependent Feature Y (price).
- Drop the target column from X.
- Print the shape of X and Y

Answer

The dataset was loaded successfully. The independent features were stored in **X** after removing the **price** column, and the target feature **price** was stored in **Y**. The shapes of both variables were printed.

PROGRAM SCREENSHOT :

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
import joblib
```

```
from google.colab import files
```

```
uploaded = files.upload()
```

```
df = pd.read_csv("ford.csv")
```

```
df.head()
```

... No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving ford.csv to ford.csv

	model	year	price	transmission	mileage	fuelType	tax	mpg	engineSize
0	Fiesta	2017	12000	Automatic	15944	Petrol	150	57.7	1.0
1	Focus	2018	14000	Manual	9083	Petrol	150	57.7	1.0
2	Focus	2017	13000	Manual	12456	Petrol	150	57.7	1.0
3	Fiesta	2019	17500	Manual	10460	Petrol	145	40.3	1.5
4	Fiesta	2019	16500	Automatic	1482	Petrol	145	48.7	1.0

```
#Q1
```

```
# Independent Features
```

```
X = df.drop("price", axis=1)
```

```
# Dependent Feature
```

```
y = df["price"]
```

```
print("Shape of X :", X.shape)
```

```
print("Shape of Y :", y.shape)
```

... Shape of X : (17966, 8)

Shape of Y : (17966,)

Q2. One-Hot Encoding)

Identify all categorical columns in X.

- Perform One-Hot Encoding using `pd.get_dummies()`.
- Convert the encoded data to integer type (`astype(int)`).
- Show the first 5 rows after encoding.

Answer

The categorical columns were identified and converted into numerical values using One-Hot Encoding. The encoded dataset was successfully converted to integer type and the first five rows were displayed.

PROGRAM SCREENSHOT :

```
#Q2

# Find categorical columns
cat_cols = X.select_dtypes(include="object").columns

print("Categorical Columns:")
print(cat_cols)

# One-Hot Encoding
X = pd.get_dummies(X, columns=cat_cols)

# Convert Boolean columns to Integer
X = X.astype(int)

print("\nEncoded Data:")
print(X.head())
```

*** Categorical Columns:

Index(['model', 'transmission', 'fuelType'], dtype='object')

Encoded Data:

	year	mileage	tax	mpg	engineSize	model_ B-MAX	model_ C-MAX	\
0	2017	15944	150	57	1	0	0	
1	2018	9083	150	57	1	0	0	
2	2017	12456	150	57	1	0	0	
3	2019	10460	145	40	1	0	0	
4	2019	1482	145	48	1	0	0	

	model_ EcoSport	model_ Edge	model_ Escort	...	model_ Transit Tourneo	\
0	0	0	0	...		0
1	0	0	0	...		0
2	0	0	0	...		0
3	0	0	0	...		0
4	0	0	0	...		0

	model_Focus	transmission_Automatic	transmission_Manual	\
0	0		1	0
1	0		0	1
2	0		0	1
3	0		0	1
4	0		1	0

	transmission_Semi-Auto	fuelType_Diesel	fuelType_Electric	\
0		0		0
1		0		0
2		0		0
3		0		0
4		0		0

	fuelType_Hybrid	fuelType_Other	fuelType_Petrol
0	0	0	1
1	0	0	1
2	0	0	1
3	0	0	1
4	0	0	1

[5 rows x 37 columns]

Q3. Feature Scaling)

Apply Standard Scaling on the numerical columns (year, mileage, tax, mpg, engine Size).

- Use Standard Scaler from sklearn. preprocessing.
- Show the first 5 rows of the scaled features.

Answer

The numerical features (**year, mileage, tax, mpg, engine Size**) were standardized using **Standard Scaler**, resulting in features with mean 0 and standard deviation 1.

PROGRAM SCREENSHOT :

```
#Q3

# Reload original dataset
df = pd.read_csv("ford.csv")

X = df.drop("price", axis=1)
y = df["price"]

# One-Hot Encoding
X = pd.get_dummies(X)

# Numerical Columns
num_cols = ["year", "mileage", "tax", "mpg", "engineSize"]

# Standard Scaling
scaler = StandardScaler()

X[num_cols] = scaler.fit_transform(X[num_cols])

print(X.head())
```

```

...      year    mileage      tax      mpg  engineSize  model_ B-MAX  \
0  0.065128 -0.380998  0.591358 -0.020442  -0.811386      False
1  0.552866 -0.733359  0.591358 -0.020442  -0.811386      False
2  0.065128 -0.560132  0.591358 -0.020442  -0.811386      False
3  1.040605 -0.662640  0.510727 -1.738890   0.345070      False
4  1.040605 -1.123724  0.510727 -0.909294  -0.811386      False

      model_ C-MAX  model_ EcoSport  model_ Edge  model_ Escort  ...  \
0          False          False      False      False      False  ...
1          False          False      False      False      False  ...
2          False          False      False      False      False  ...
3          False          False      False      False      False  ...
4          False          False      False      False      False  ...

      model_ Transit Tourneo  model_Focus  transmission_Automatic  \
0          False          False          True
1          False          False          False
2          False          False          False
3          False          False          False
4          False          False          True

      transmission_Manual  transmission_Semi-Auto  fuelType_Diesel  \
0          False          False          False
1          True          False          False
2          True          False          False
3          True          False          False
4          False          False          False

      transmission_Manual  transmission_Semi-Auto  fuelType_Diesel  \
0          False          False          False
1          True          False          False
2          True          False          False
3          True          False          False
4          False          False          False

      fuelType_Electric  fuelType_Hybrid  fuelType_Other  fuelType_Petrol
0          False          False          False          True
1          False          False          False          True
2          False          False          False          True
3          False          False          False          True
4          False          False          False          True

```

[5 rows x 37 columns]

Q4. Train-Test Split)

Split the preprocessed data into training and testing sets using `train_test_split`.

- Use `test_size=0.33` and `random_state=42`.
- Print the shape of `X_train`, `X_test`, `y_train`, and `y_test`.

Answer

The dataset was split into **67% training data** and **33% testing data** using `train_test_split()` with `random_state=42`.

PROGRAM SCREENSHOT :

```
#Q4

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.33,
    random_state=42
)

print("X_train :", X_train.shape)
print("X_test  :", X_test.shape)
print("y_train :", y_train.shape)
print("y_test  :", y_test.shape)
```

```
... X_train : (12037, 37)
    X_test  : (5929, 37)
    y_train : (12037,)
    y_test  : (5929,)
```

Q5. Building Linear Regression Model)

Train a **Linear Regression** model using sklearn.

- Fit the model on the training data X_train, y_train).
- Print the intercept and coefficients of the model.

Answer

The Linear Regression model was trained successfully. The intercept and coefficients were generated, showing how each feature affects the predicted car price.

PROGRAM SCREENSHOT :

```
#Q5

model = LinearRegression()

model.fit(X_train, y_train)

print("Intercept:")
print(model.intercept_)

print("\nCoefficients:")
print(model.coef_)
```



```
Intercept:  
16624.42763578743
```

```
Coefficients:  
[ 2.14740256e+03 -1.32532970e+03 -6.27759124e+01 -8.20353166e+02  
 1.08356730e+03 -4.02639412e+03 -2.92796686e+03 -2.35926570e+03  
 5.53724546e+03 9.97603894e+03 -2.84721223e+03 -7.96862146e+02  
 -3.36031565e+02 2.56961644e+03 -2.53006215e+03 5.09726763e+01  
 -4.25957681e+03 -6.87550273e+03 -2.82962359e+02 -1.03438708e+03  
 8.20550001e+03 3.71333848e+03 1.59161573e-11 2.37523965e+03  
 4.54747351e-13 -4.98969646e+02 2.69783164e+03 -2.30983633e+03  
 -4.04075355e+03 1.84573692e+02 -2.41088798e+02 5.65151064e+01  
 -2.16206602e+03 0.00000000e+00 5.99557104e+03 -1.26539633e+03  
 -2.56810869e+03]
```

Q6. Making Predictions)

Use the trained model to make predictions on the test data.

- Store predictions in `y_pred`.
- Print the first 10 predicted values and compare them with the first 10 actual values from `y_test`.

Answer

Predictions were made using the trained model. The first ten predicted values were compared with the actual values from the test dataset. The predicted prices were close to the actual prices, indicating good model performance.

PROGRAM SCREENSHOT :

```
#Q6

y_pred = model.predict(X_test)

print("First 10 Predicted Prices")
print(y_pred[:10])

print("\nFirst 10 Actual Prices")
print(y_test.iloc[:10].values)
```

```
... First 10 Predicted Prices
[ 6676.74792864  9175.10363172  9876.15109278  3808.41679254
 2944.77931074 16623.10788236 27571.14532174 12128.40953688
11577.4693611 16774.32274028]

First 10 Actual Prices
[ 6995  8999  7998  5491  3790 18200 22998 11000  7600 14985]
```

Q7. Model Evaluation using R^2 Score)

Calculate the R^2 score using `r2_score(y_test, y_pred)`.

- Interpret the R^2 value in your own words.
- Comment on how well the model is performing.

Answer:

The R^2 score obtained is **0.93** (or the value you get after running the code). This means the model explains about **93%** of the variation in car prices. Since the R^2 score is close to **1**, the model performs very well and provides accurate predictions.

PROGRAM SCREENSHOT :

```
#Q7

r2 = r2_score(y_test, y_pred)

print("R2 Score =", r2)

R2 Score = 0.8408137113174078
```

Q8. Saving the Model)

Save your trained model and preprocessing objects using joblib:

- The Linear Regression model → "LR_ford_car.pkl"
- The StandardScaler → "scaler.pkl"
- The list of columns → "columns.pkl"

Answer

The trained Linear Regression model, the StandardScaler object, and the feature column names were successfully saved using **joblib**. These files can be loaded later to make predictions without retraining the model.

PROGRAM SCREENSHOT :

```
#Q8

joblib.dump(model, "LR_ford_car.pkl")
joblib.dump(scaler, "scaler.pkl")
joblib.dump(X.columns.tolist(), "columns.pkl")

print("All files saved successfully.")

All files saved successfully.
```

- **Final Conclusion:**

The Ford Car dataset was successfully preprocessed by applying One-Hot Encoding and Standard Scaling. The data was split into training and testing sets, and a Linear Regression model was trained. Predictions were made on the test data, and the model was evaluated using the **R² Score**, which indicated good performance. Finally, the trained model, scaler, and feature columns were saved using **joblib** for future use.